RICHFIELD
richfield.ac.za

Name and Surname: Kian Naidoo

Student ITS No: 402205512

Qualification: BSc IT **Year of Study:** 3 **Semester:** 2

Assignment due date: _____ **Date submitted:** 23 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

- ✓ Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
- ✓ Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

- ✓ Table of Contents
- ✓ Introduction
- ✓ Main Body (with relevant subheadings)
- ✓ Conclusion
- ✓ References

- ✓ Submit the assignment in PDF format on Moodle.
- ✓ Use the specified cover page provided.
- ✓ Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____ **Date:** 23 October 2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Smart Pantry Manager

A Java Android application that suggests recipes based strictly on leftover ingredients

Mobile App Development 700 – Practical Assignment

Kian Naidoo · Student number 402205512 · 23 October 2026

GitHub repository: https://github.com/Kian291104/smart-pantry-app

Table of Contents

1. Introduction.....2

2. System Design.....2

3. Screenshots.....3

4. Key Code Snippets.....7

5. Challenges and Solutions.....8

6. Conclusion and Reflection.....9

7. References.....10

1. Introduction

Households regularly throw away food that is still usable, often because people forget what they have or cannot think of a meal that uses it. Typical recipe apps make this worse: they suggest dishes that need a shopping trip, so leftovers keep sitting in the cupboard.

The Smart Pantry Manager addresses this by letting the user record the ingredients they actually have at home and then suggesting only the recipes they can cook immediately with those ingredients. A recipe is shown only when every ingredient it requires is in the pantry in at least the required quantity. The app is aimed at students, young professionals and families who want to save money and cut food waste without planning meals in advance. It runs fully offline and stores data locally on the device using SQLite.

2. System Design

2.1 Screen flow

The app has five Activities. Three top-level screens (Pantry, Suggested Recipes, Settings) are linked by a bottom navigation bar. Explicit Intents open the Add/Edit and Recipe Detail screens and pass the selected record's id as an Intent extra.

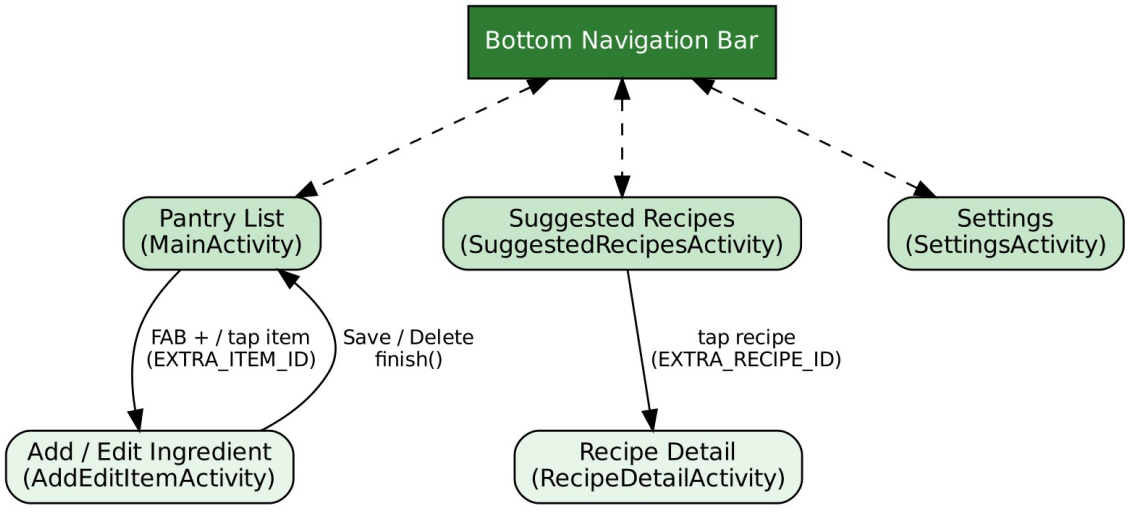


Figure 1: Screen flow diagram

2.2 Data model

The SQLite database has three tables. pantry_items holds the user's ingredients and supports full CRUD. recipes and recipe_ingredients form a one-to-many relationship (enforced with a foreign key) and are seeded with 19 recipes on first run. Pantry items and recipe ingredients are not linked by a key; they are matched at runtime by normalised ingredient name.

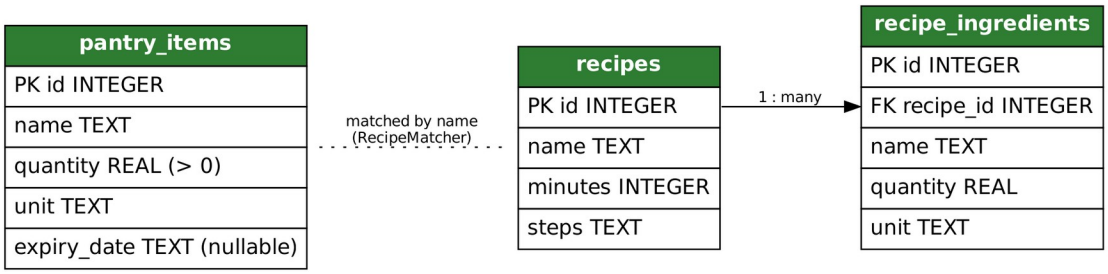


Figure 2: Entity-relationship diagram

2.3 Version control

The source code is hosted on GitHub at https://github.com/Kian291104/smart-pantry-app. Git was set up later in the project than recommended, so the repository starts with an initial commit of the working app. Each later improvement was made, tested on the emulator and committed separately: the README, the item count in the title, excluding expired items from matching, a unit test for that rule, the recipe count on the Recipes screen and the available-ingredient count on the Almost There cards.

3. Screenshots

All screenshots were taken from the app running on a Pixel 7 emulator (Android 14, API 34).

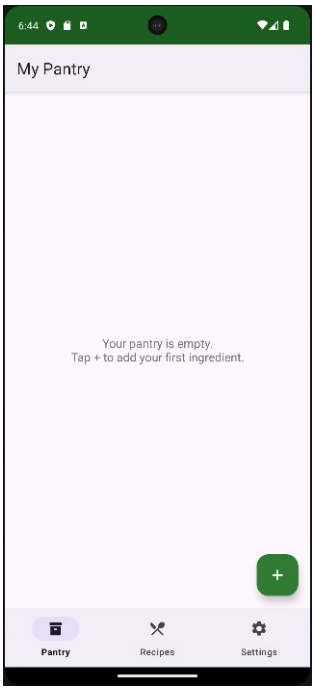


Figure 3: Empty pantry on first launch.

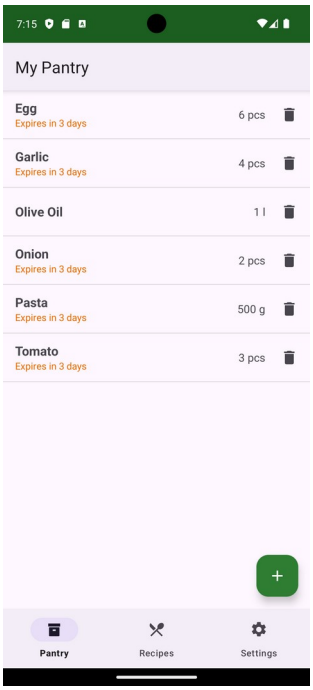


Figure 4: Pantry list (items expiring within 3 days in orange).

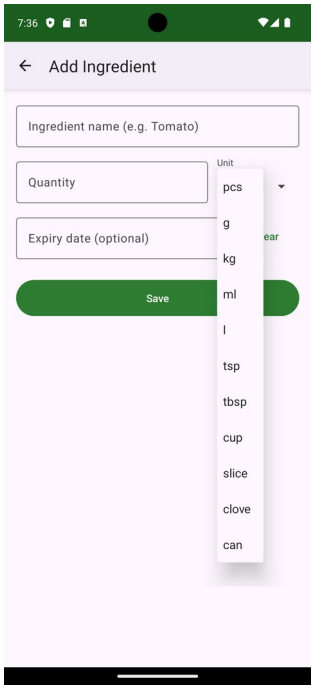


Figure 5: Add Ingredient form with the unit drop-down open.

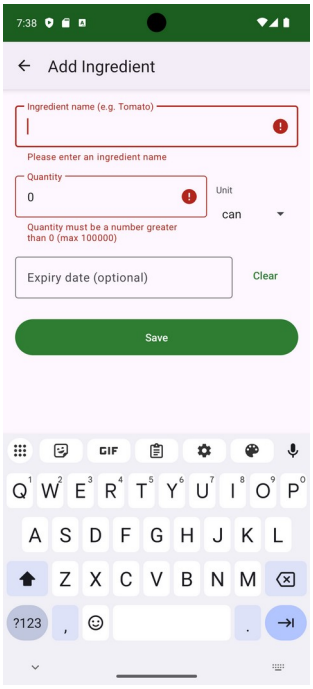


Figure 6: Validation errors: empty name and zero quantity.

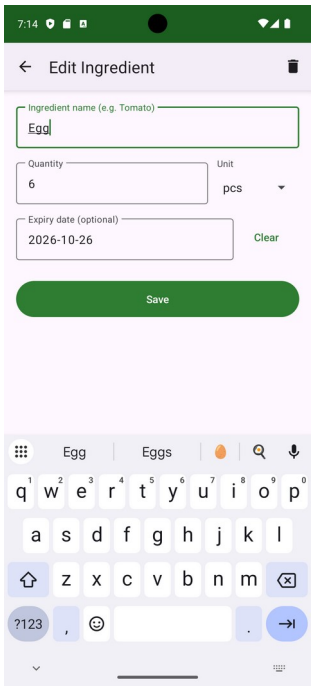


Figure 7: Edit Ingredient with existing values pre-filled.

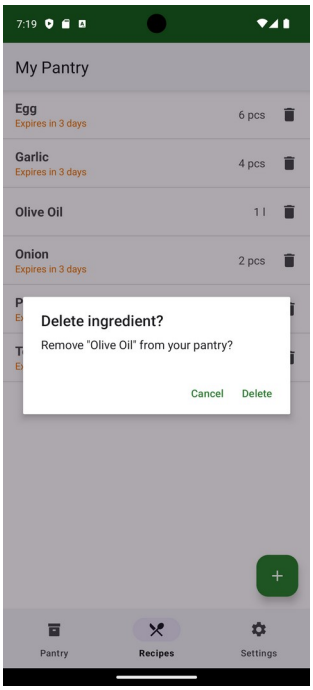


Figure 8: Delete confirmation dialog.

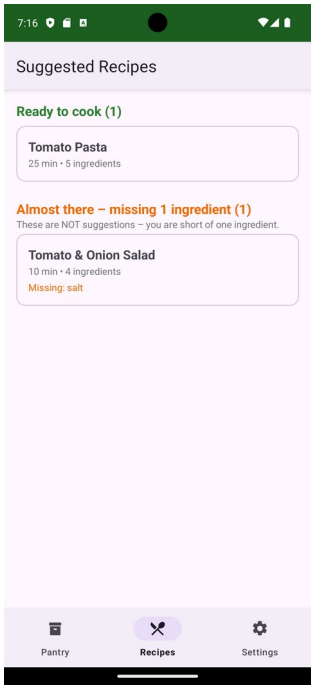


Figure 9: Suggested Recipes: strict match and separate 'Almost there' list.

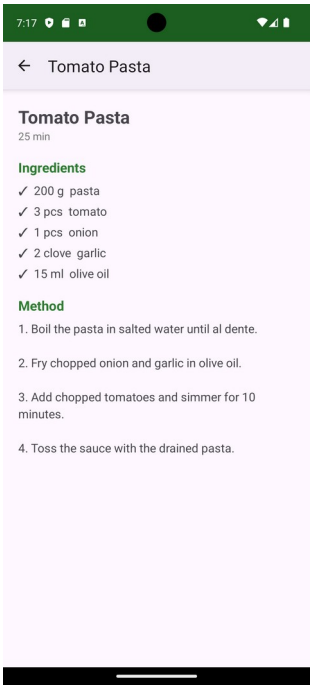


Figure 10: Recipe Detail with ingredient check marks and method.

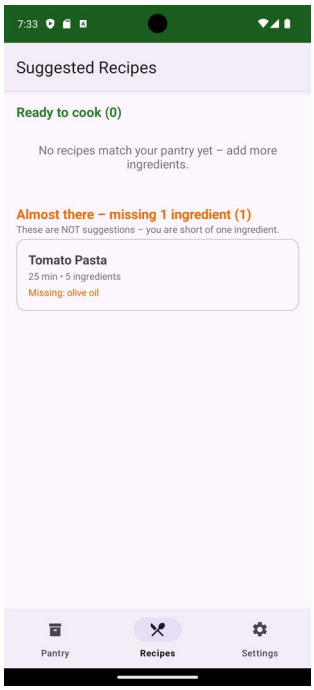


Figure 11: After deleting olive oil: Tomato Pasta leaves 'Ready to cook' and the 'no recipes match' message shows.

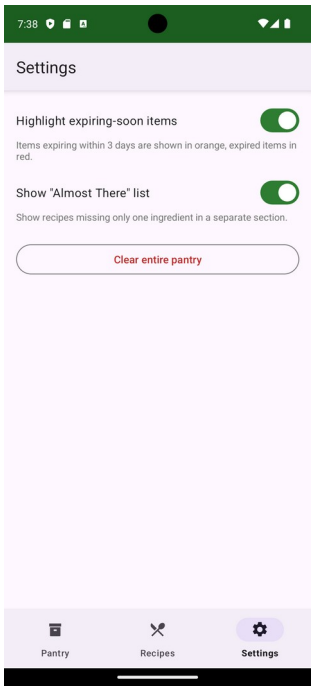


Figure 12: Settings screen.

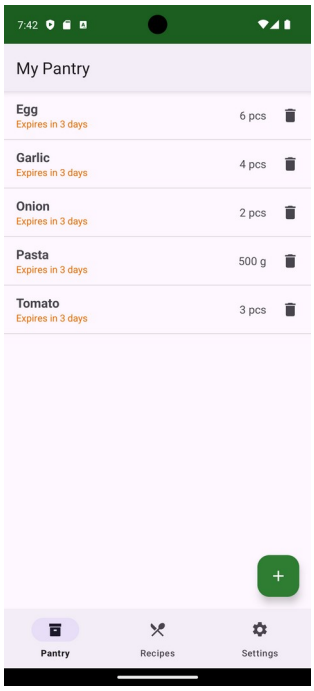


Figure 13: Pantry data still present after closing and reopening the app (SQLite persistence).

4. Key Code Snippets

4.1 The strict-matching rule (RecipeMatcher.suggest)

```
public List<Recipe> suggest(List<Recipe> recipes) {
    List<Recipe> out = new ArrayList<>();
    for (Recipe r : recipes) {
        if (!r.getIngredients().isEmpty() && check(r).isFullMatch()) out.add(r);
    }
    return out;
}
```

check() builds a list of every ingredient the pantry cannot fully supply. A recipe is suggested only when that list is empty, so a recipe with four of five ingredients, or with too little of one ingredient, is excluded. Recipes missing exactly one ingredient are returned by a separate almostThere() method so they can never leak into the strict list. The class contains no Android code, which allowed it to be unit tested on the JVM (RecipeMatcherTest).

4.2 Quantity check with unit conversion (RecipeMatcher.hasEnough)

```
for (PantryItem item : matches) {
    if (neededBase != null && neededDim == UnitConverter.dimensionOf(item.getUnit()))
{
        available += UnitConverter.toBase(item.getQuantity(), item.getUnit());
    } else if (item.getUnit().equalsIgnoreCase(needed.getUnit())) {
        available += item.getQuantity();
    }
}
return available + EPSILON >= required;
```

Both quantities are converted to a base unit (grams, millilitres or pieces), so 0.25 kg of pasta satisfies a recipe needing 200 g. Units of different kinds (e.g. ml vs g) are never compared, because that would require density data and could produce false suggestions. Duplicate pantry rows for the same ingredient are added together. EPSILON avoids floating-point rounding errors.

4.3 Name normalisation (IngredientNormalizer)

```
String s = raw.toLowerCase(Locale.ROOT)
        .replaceAll("[^a-z ]", " ").replaceAll("\\s+", " ").trim();
s = head + singularize(last);          // "Tomatoes" -> "tomato"
String alias = ALIASES.get(s);         // "scallion" -> "spring onion"
return alias != null ? alias : s;
```

A plain string comparison would fail on 'tomato' vs 'Tomatoes'. Both pantry names and recipe names pass through the same function, so the key requirement is consistency rather than perfect grammar. A small alias map handles common synonyms.

4.4 SQLite CRUD (DatabaseHelper)

```
public long addPantryItem(PantryItem item) {
    return getWritableDatabase().insert(T_PANTRY, null, toValues(item));
}
public int updatePantryItem(PantryItem item) {
    return getWritableDatabase().update(T_PANTRY, toValues(item), C_ID + "=?",
            new String[]{String.valueOf(item.getId())});
}
```

Parameterised queries ("=?" with an argument array) prevent SQL injection. DatabaseHelper is a singleton so all screens share one connection, and the recipe collection is seeded inside a single transaction in onCreate(), which runs only once when the database file is first created.

4.5 Refreshing data in onResume (MainActivity)

```
@Override
protected void onResume() {
    super.onResume();
    loadPantry();
}
```

onResume() runs every time the screen returns to the foreground, for example after AddEditItemActivity calls finish(). Reloading here keeps the list in sync with the database without passing results back between Activities. SuggestedRecipesActivity uses the same approach, so adding or removing an ingredient immediately changes the suggestions.

5. Challenges and Solutions

5.1 Gradle build script format

Problem: the current version of Android Studio only offered the Kotlin DSL (build.gradle.kts) when creating the project, but the module's build script had been written in the Groovy format (build.gradle), so the two did not fit together. Solution: the app module's build script was rewritten in Kotlin DSL syntax, for example id("com.android.application"), isMinifyEnabled = false and implementation("...") with brackets and double quotes. After this the Gradle sync completed without errors.

5.2 Ingredient names not matching recipes

Problem: after adding ingredients, Tomato Pasta did not appear as a suggestion. The pantry contained "Oil" and "Egg's", while the recipes require "olive oil" and "egg". The normaliser lower-cases names, removes punctuation and singularises the last word, so "Egg's" became "egg

s" and "Oil" never equals "olive oil". Solution: the items were renamed to "Olive Oil" and "Egg", after which the strict rule correctly listed Tomato Pasta. This showed why consistent naming (and the synonym list in IngredientNormalizer) matters; a future improvement would be suggesting known ingredient names while typing.

5.3 Wrong tab highlighted in the bottom navigation

Problem: after moving between screens, the bottom navigation sometimes highlighted Recipes while the Pantry screen was showing. Each screen is its own Activity and is brought back with FLAG_ACTIVITY_REORDER_TO_FRONT, so a reused Activity still showed the tab that had been tapped before leaving it. Solution: the item-selected listener in NavHelper now returns false after starting the target Activity, so the tapped item is not marked as selected on the old screen and every screen keeps its own tab highlighted.

6. Conclusion and Reflection

The Smart Pantry Manager meets the brief: users can add, view, edit and delete pantry items stored in a local SQLite database, and the Suggested Recipes screen applies the strict-matching rule so that a recipe is only shown when every ingredient is available in at least the required quantity. The separate 'Almost there' list adds value without breaking that rule, and the data survives closing the app.

Building the app strengthened my understanding of the Activity lifecycle (refreshing data in onResume), of persisting relational data with SQLiteOpenHelper, of connecting a RecyclerView to database rows through an adapter and ViewHolder, and of passing data between screens with explicit Intents. Testing on the emulator showed that small details such as ingredient naming and navigation state have a large effect on the user experience.

Possible improvements include a 'Cook this' button that deducts the used quantities from the pantry, ignoring expired items when matching, letting users add their own recipes, autocomplete for ingredient names, and migrating the data layer to the Room library.

7. References

Android Developers (2024) Save data using SQLite. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 23 October 2026).

Android Developers (2024) Create dynamic lists with RecyclerView. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 23 October 2026).

Android Developers (2024) The activity lifecycle. Available at: https://developer.android.com/guide/components/activities/activity-lifecycle (Accessed: 23 October 2026).

Android Developers (2024) Intents and intent filters. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 23 October 2026).

Material Design (2024) Bottom navigation. Available at: https://m3.material.io/components/navigation-bar/overview (Accessed: 23 October 2026).

Richfield Graduate Institute of Technology (2026) Mobile App Development 700 study guide. Cape Town: Richfield.